

Basketball reference has an excel download option for each season player stats page. Here is what it looks like in the raw data download:

Here is an example of the raw spreadsheet download. Notice “James Harden” rank 22. This was an initial data issue I ran into. Players who played on multiple teams for a season had multiple entries. I need to make sure only their first entry was included, as their whole season stats.


```

if not duplicates.empty:
    print(f"Duplicates found in {season}:")
    # Print the player names and the rows
    print(duplicates[['Player']])
else:
    print(f"No duplicates found in {season}.")

```

```
Duplicates found in 2025-26:
```

```

      Player
21    James Harden
22    James Harden
23    James Harden
64      Trae Young
65      Trae Young
..      ...
420  Guerschon Yabusele
421  Guerschon Yabusele
438      Rayan Rupert
439      Rayan Rupert
440      Rayan Rupert

```

```
[69 rows x 1 columns]
```

```
No duplicates found in 2024-25.
```

```
No duplicates found in 2023-24.
```

```
No duplicates found in 2022-23.
```

```
No duplicates found in 2021-22.
```

```
No duplicates found in 2020-21.
```

```
No duplicates found in 2019-20.
```

```
Duplicates found in 2018-19:
```

```

      Player
32  Tobias Harris
33  Tobias Harris
34  Tobias Harris
41  Jimmy Butler
42  Jimmy Butler
..      ...

```

```
688  Ron Baker
```

```
689  Ron Baker
```

```
694  Jawun Evans
```

```
695  Jawun Evans
```

```
696  Jawun Evans
```

SOURCE

KERNEL SOURCES

Would you like to get notified about official Jupyter news?

[Open privacy policy](#)

Yes

No

#Keep the first player entry as that will be the entry with the total for their multiple teams.

```
for season, df in dfs.items():
```

```
    df.drop_duplicates(subset=['Player'], keep='first', inplace=True)
```

```
    print(f"Duplicates removed from {season}")
```

```
print(f"Duplicates removed from {season}")
```

```
Duplicates removed from 2025-26
```

```
Duplicates removed from 2024-25
```

```
Duplicates removed from 2023-24
```

```
Duplicates removed from 2022-23
```

```
Duplicates removed from 2021-22
```

```
Duplicates removed from 2020-21
```

```
Duplicates removed from 2019-20
```

```
Duplicates removed from 2018-19
```

```
Duplicates removed from 2017-18
```

```
Duplicates removed from 2016-17
```

```
Duplicates removed from 2015-16
```

```
[19]: #Lebron James season by season scores with his overall rank
```

Next I wanted to compare Lebron James season by season score and rank(remember the score is in comparison to the highest player in the league so a more dominant top player will result in lower scores).

```

#Lebron James season by season scores with his overall rank
import matplotlib.pyplot as plt

# 1. Initialize a list to hold data from each season
plot_data = []

# 1. Process all seasons to get LeBron's Rank and Score
for season, df in dfs.items():
    # Calculate rank for everyone in the league for this season
    df['Rank'] = df['Mayscore to 100'].rank(method='dense', ascending=False)

    # Filter for LeBron
    lebron = df[df['Player'].str.contains('LeBron James', case=False, na=False)]

    if not lebron.empty:
        score = lebron['Mayscore to 100'].iloc[0]
        rank = int(lebron['Rank'].iloc[0])
        plot_data.append({'Season': season, 'Score': score, 'Rank': rank})

# 2. Create DataFrame and sort by Season
df_plot = pd.DataFrame(plot_data).sort_values('Season')

# 3. Plotting
fig, ax = plt.subplots(figsize=(12, 6))

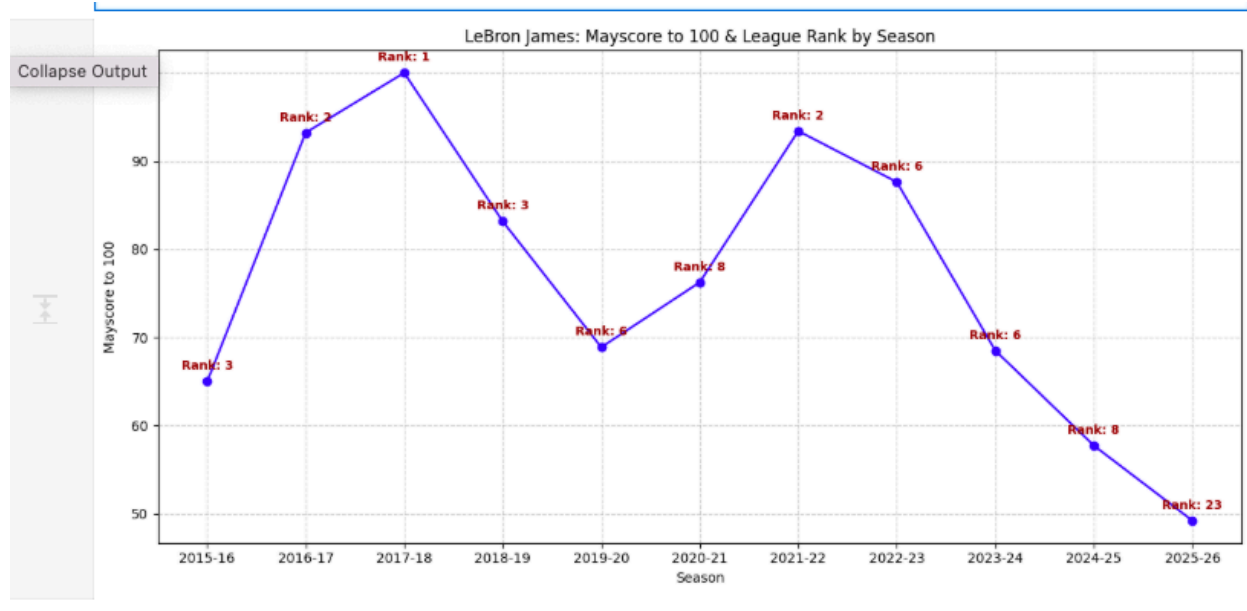
# Plot the main line of mayscore to 100
ax.plot(df_plot['Season'], df_plot['Score'], marker='o', linestyle='-', color='blue', label='Mayscore')

# 4. Add Overall Player Rank
for i, row in df_plot.iterrows():
    ax.text(row['Season'], row['Score'] + 1, f"Rank: {row['Rank']}",
            ha='center', va='bottom', fontsize=9, fontweight='bold', color='darkred')

ax.set_title('LeBron James: Mayscore to 100 & League Rank by Season')
ax.set_ylabel('Mayscore to 100')
ax.set_xlabel('Season')
ax.grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()

```



The next thing I went to analyze was the overall health of the league by seeing how many players didnt meet the 25 game threshold each season:

#Players who did not qualify by games played each season. Also will show health of the league.

import seaborn as sns

ineligible_counts = []

for season, df in dfs.items():

 # Count rows where the column equals 'Not Enough Games Played'

 if 'Enough Games Played?' in df.columns:

 count = df[df['Enough Games Played?'] == 'Not Enough Games Played'].shape[0]

 ineligible_counts.append({'Season': season, 'Count': count})

 else:

 print(f"Column 'Enough Games Played?' not found in {season}")

2. Create a DataFrame for plotting

df_counts = pd.DataFrame(ineligible_counts).sort_values('Season')

3. Plotting the results

plt.figure(figsize=(10, 6))

plt.bar(df_counts['Season'], df_counts['Count'], color='orange', edgecolor='black')

plt.title('Number of Players Not Meeting 25 Game Requirements per Season')

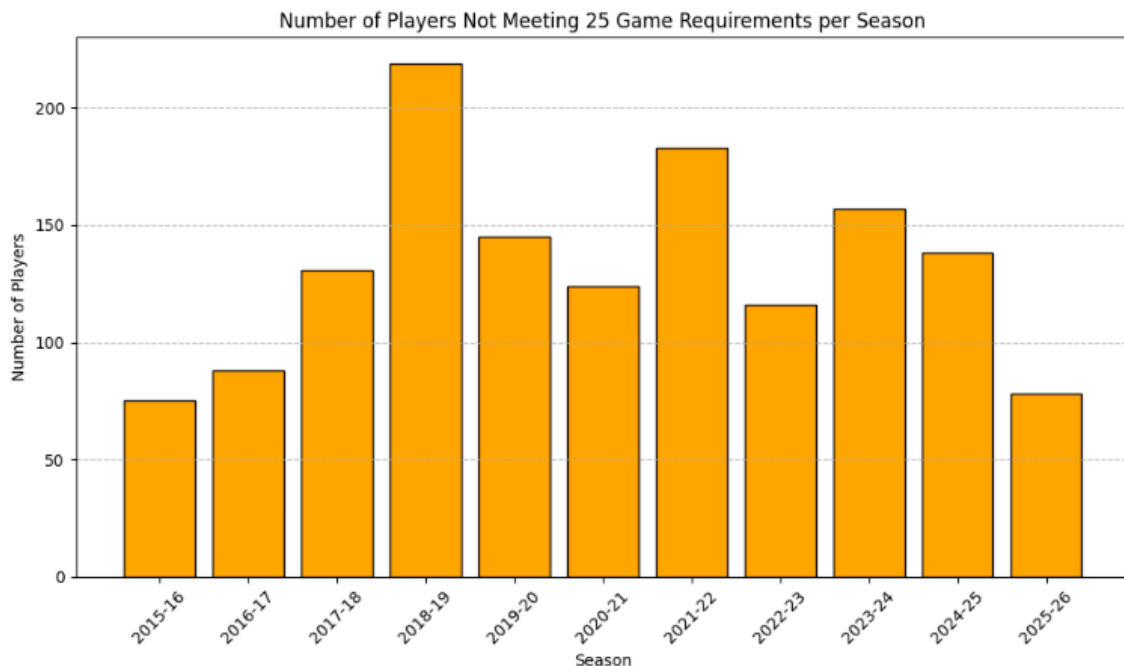
plt.xlabel('Season')

plt.ylabel('Number of Players')

plt.xticks(rotation=45)

plt.grid(axis='y', linestyle='--', alpha=0.7)

```
plt.tight_layout()
plt.show()
```



Then lastly I wanted to compare the last 11 “Mayscore” MVPs versus the actual league MVPs:
 #This step I am importing the last 11 MVPs into the data frame and comparing the actual MVP
 vs the Mayscore MVP

```
mvp_data = {
    'Season': ['2015-16', '2016-17', '2017-18', '2018-19', '2019-20', '2020-21',
              '2021-22', '2022-23', '2023-24', '2024-25', '2025-26'],
    'Actual_MVP': ['Stephen Curry', 'Russell Westbrook', 'James Harden', 'Giannis
Antetokounmpo',
                  'Giannis Antetokounmpo', 'Nikola Jokic', 'Nikola Jokic', 'Joel Embiid',
                  'Nikola Jokic', 'Shai Gilgeous-Alexander', 'Shai Gilgeous-Alexander']
}
df_mvp = pd.DataFrame(mvp_data)
```

2. Plotting

```
plt.figure(figsize=(14, 6))
```

Plot the Mayscore Leaders

```
plt.scatter(df_top['Season'], [100] * len(df_top), color='blue', s=100, label='Mayscore Leader')
```

Plot the Actual MVPs

```
plt.scatter(df_mvp['Season'], [99.8] * len(df_mvp), color='red', marker='X', s=100, label='Actual
MVP')
```

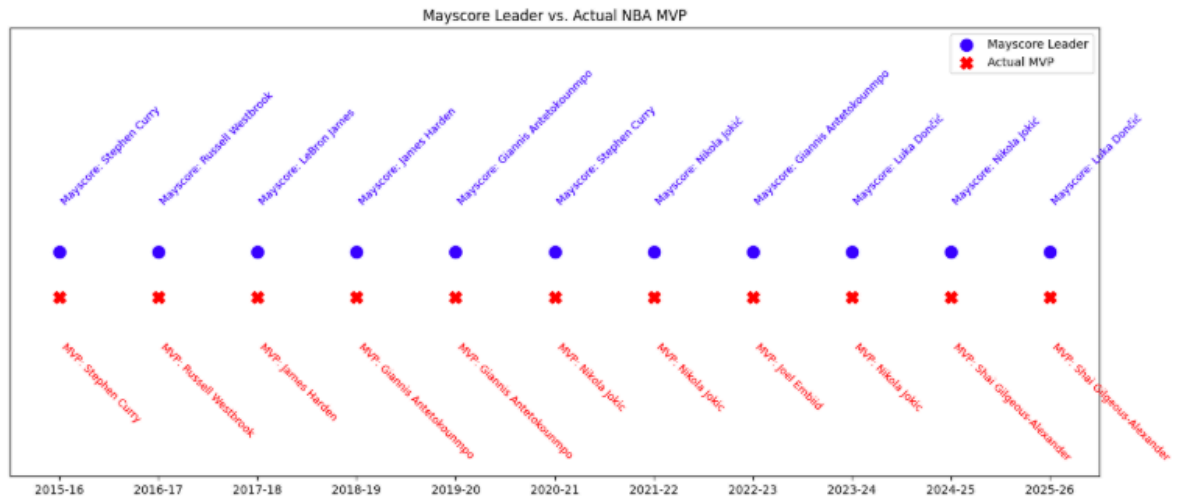
```

# Add Labels
for i, row in df_top.iterrows():
    plt.text(row['Season'], 100.2, f'Mayscore: {row["Player"]}',
             rotation=45, ha='left', va='bottom', fontsize=9, color='blue')

for i, row in df_mvp.iterrows():
    plt.text(row['Season'], 99.6, f'MVP: {row["Actual_MVP"]}',
             rotation=-45, ha='left', va='top', fontsize=9, color='red')

plt.title('Mayscore Leader vs. Actual NBA MVP')
plt.ylim(99, 101)
plt.yticks([]) # Hide Y-axis as it's just a timeline
plt.legend()
plt.tight_layout()
plt.show()

```



In conclusion, my metrics predicted 4/11 past MVPs. The actual MVP is a very subjective thing each season as it is voted heavily by the media. Often team wins are a major factor in the MVP along with individual stats.